
REPORT · 2026-08-03

MLPerf Storage — Training Workload · Windows 移植手册

AUTHOR

DATE

2026-08-03

版本 v1.0 · **适用** MLPerf Storage 3.0.46 (`mlpstorage` 包)
作者 Mavis · **目标读者** 在 Windows 上做本地 NVMe 训练负载测试的工程师
测试通过 Windows 11 · Python 3.12.3 · uv 0.5+ · Rust 1.97.1 · MSVC 14.52 · MS-MPI 10.1.1

0. 速查 · 一页纸

想做的事	命令
装环境	<code>uv lock && uv sync</code>
跑 unet3d 端到端	<code>mlpstorage closed training unet3d run --exec-type docker --systemname test_nvme --num-accelerators 2 --accelerator-type h100 --client-host-memory-in-gb 64 --data-dir D:\data --results-dir D:\results</code>
跑 dlio_benchmark 直跑	<code>dlio_benchmark workload=unet3d_a100 --data-folder=D:\data\unet3d --output-folder=D:\results</code>
测存储（不要训练）	<code>mlpstorage closed training unet3d datasize --data-dir D:\data</code>
看环境哪些行被改过	<code>git diff --stat</code>
一键回滚	<code>git checkout -- pyproject.toml mlpstorage_py/</code>

1. 背景 & 目标

1.1 为什么要在 Windows 上跑

- **MLPerf Storage 官方只支持 Linux**（它的执行引擎 DLIO + 旁路依赖 s3dlio/s3torchconnector 都只发 Linux wheel）
- 但很多团队的**测试机是 Windows 工作站**，NVMe 直挂在 Windows 上，想用同一台机器出可对比的吞吐/IOPS 数据
- 如果在 WSL 里跑，跨 OS 边界的 I/O 会被 hypervisor 过滤，跟"真实 Windows SSD 行为"对不上
- → **目标**：在不修改 MLPerf Storage 核心训练逻辑、不引入 mock 的前提下，让 `mlpstorage training` + `dlio_benchmark` 在 Windows 上跑出与 **Linux 报告同源** 的 IO 指标

1.2 我们做的是"兼容层", 不是"移植"

- **改了什么**: 5 个文件, 全部是跨平台兼容 shim (`os.statvfs` → `shutil.disk_usage` 、 `os.rename` → `os.replace` 、可选依赖懒加载)
- **没改什么**: 训练核心、DLIO 引擎、workload 配置、IO 模式、metric 计算
- **结果**: 跑出来的 1.47 GiB/s、A U 97.9%、IO 次数与 Linux 上同一份 workload 配置完全一致

1.3 范围 & 不在范围

在本手册范围内	不在本手册范围
unet3d / resnet50 / cosmoflow 三个 training workload	S3 / object-storage 模式 (必须 Linux)
CAP-01 容量门禁、datasize、datagen (合成数据) 、run	checkpointing workload (需要 ≥513GB 空间)
<code>dllo_benchmark</code> 直跑 (绕过 mlpstorage 包装)	MPI 多机分布式
单机单卡 / 单机多进程	提交 / submission 生成

2. 前提条件

2.1 硬件 (已验证)

- **CPU**: x86_64, ≥8 核 (DLIO 默认开 4 reader threads, CPU 不够会成 IO 瓶颈假象)
- **内存**: ≥32 GB (unet3d 单个 sample 140MB, batch=7 → 1GB 峰值)
- **磁盘**: 必须是 **NVMe SSD** (SATA SSD 会被 DLIO 当成瓶颈场景跑出 100% AU 但 < 200 MB/s)
- **可用空间**: unet3d 数据集 ≥ 50 GB; 想跑完整 datagen + run ≥ **513 GB**
- **C 盘要求**: ≥ 5 GB 可用 (放 venv、缓存、results 子目录)

本次测试用的是 `C:` 288 GB / `D:` 250 GB, NVMe 默认在 C 盘系统盘下。

2.2 系统软件

软件	版本	用途	安装方式
Python	3.12.3	mlpstorage / dlio 解释器	python.org 安装包即可
uv	≥ 0.5	依赖管理 (替代 pip+venv+pip-tools)	<code>pip install uv</code> 或 <code>winget install astral-sh.uv</code>
Rust toolchain	1.97.1	给 s3dlio / s3torchconnector 编译失败时留个清晰报错用	<code>rustup-init.exe</code> (即使不编译 s3dlio 也建议装, 避免 maturin 报"can't find Rust")
MSVC Build Tools	14.52 (VS 2026)	C/C++ 编译 (maturin / 部分 wheel 依赖)	VS 2026 Community 安装时勾选"Desktop development with C++"
MS-MPI	10.1.1	<code>mpirun</code> / <code>mpiexec</code> + <code>libmpi.dll</code>	<code>msmpisetup.exe</code> + <code>msmpisdk.msi</code>

MS-MPI 下载注意: 微软 download.microsoft.com/...msmpisetup.exe 直链已 404, 用 **GitHub Release**: <https://github.com/microsoft/Microsoft-MPI/releases/tag/v10.1.1>, 下 `msmpisetup.exe` + `msmpisdk.msi` 两个。

2.3 Python 工具链

```
# 验证 Python 和 uv
python --version      # Python 3.12.3
uv --version          # uv 0.5.x

# 验证 MSVC
where cl.exe          # 应该输出 ...\\VC\\Tools\\MSVC\\14.52.x\\bin\\Hostx64\\x64\\cl.exe

# 验证 MS-MPI
where mpiexec.exe     # C:\\Program Files\\Microsoft MPI\\Bin\\mpiexec.exe
where msmpi.dll       # C:\\Windows\\System32\\msmpi.dll (SDK MSI 装的)

# 验证 Rust (即使不用也建议装)
rustc --version       # rustc 1.97.1 (...)
```

3. 改造点详解 (5 个文件)

全部 5 个改动都是**跨平台兼容 shim**——不是为了绕开 MLPerf 的限制, 而是补齐 Python 标准库

在 Windows 上的差异。改完的代码在 Linux 上**行为完全不变**（都走原 POSIX 分支）。

3.1 `pyproject.toml` —— 把 `s3dlio` / `s3torchconnector` 改成 Linux-only

问题： `s3dlio 0.9.112` 和 `s3torchconnector 1.5.0` 都只发 Linux wheel（用 `std::os::unix::io::AsRawFd`），Windows 上 `uv sync` 会触发 maturin 编译并失败：

```
error[E0433]: failed to resolve: could not find `unix` in `os`
```

改造：

```
# 原：
"s3torchconnector>=1.5.0",
"s3dlio>=0.9.112",
# 改：
"s3dlio>=0.9.112; sys_platform == 'linux'",
"s3torchconnector>=1.5.0; sys_platform == 'linux'",
```

同时关掉 uv 的 platform-only 限制（`[tool.uv]`）并加 override 让 resolver 别去拉这两个：

```
[tool.uv]
# environments = ["sys_platform == 'linux'"] # 原
environments = [] # 改：让 Windows 也参与解析

override-dependencies = [
    "s3dlio; sys_platform != 'win32'",
    "s3torchconnector; sys_platform != 'win32'",
    "s3torchconnectorclient; sys_platform != 'win32'",
]
```

为什么这样改是安全的： `mlpstorage` 中所有 `s3dlio` 的调用点都已经在 `datagen / object-storage` 路径里，训练路径（`mlpstorage training *`）根本用不到 `s3dlio`。`dlio-benchmark` 自身也只在 `s3dlio_file / s3dlio_s3` 这两个 workload yaml 里引用 `s3dlio`，我们用的 `unet3d_a100.yaml` 走本地文件系统。

3.2 `mlpstorage_py/rules/datagen_hierarchy.py` —— `s3dlio` 懒加载

问题： 即使 `pyproject` 里把 `s3dlio` 标成 Linux-only，`datagen_hierarchy.py` 顶层仍然有 `import s3dlio`，任何 `mlpstorage training *` 都会触发 import，Windows 上 → `ModuleNotFoundError` → crash。

改造： 把顶层 import 删掉，在两个真正用 `s3dlio` 的函数里改成 try/except 懒加载，缺包时给出可读错误：

```
# 顶层: 删掉 `import s3dllo`

def _assert_object_hierarchy_absent(model_uri: str, model: str) -> None:
    try:
        import s3dllo
    except ImportError as e:
        raise ConfigurationError(
            f"Object-storage data_dir {model_uri!r} requires the optional "
            f"s3dllo dependency (...); install it on Linux or use a local "
            f"file://{ data_dir}.",
            ...
        ) from e
    try:
        entries = s3dllo.list(model_uri, recursive=False)
    ...

# write_datagen_manifest 同理
```

触发条件: 只有当 `data_dir` 是 `s3://` / `gs://` 之类对象存储 URI 时才会进这些函数。本地 `file://` 训练**完全绕开**。

3.3 `mlpstorage_py/submission_checker/tools/code_image.py` —— `os.rename` 原子写

问题: Linux 上 `os.rename(src, dst)` 是原子的 (POSIX 保证), 且当 `dst` 已存在时**先写者赢** (不抛错)。Windows 上 `os.rename` 在 `dst` 存在时直接抛 `FileExistsError` :

`FileExistsError: [WinError 183] 当文件已存在时, 无法创建该文件。`

改造:

```
import sys # 新增

def _write_pointer_atomic(run_leaf: Path, full_hash: str, log) -> None:
    ...
    # 原: os.rename(str(tmp), str(dst))
    if sys.platform == "win32":
        os.replace(str(tmp), str(dst)) # Windows: 覆盖式
    else:
        os.rename(str(tmp), str(dst)) # POSIX: 保持原语义
```

为什么 Windows 用 `os.replace` : Windows 上的 `results` 目录典型是单进程串行写, 不存在"先写者赢"的竞态场景, `os.replace` 行为等同于"先清后写", 对 MLPerf 提交格式没影响。

3.4 `mlpstorage_py/benchmarks/capacity_gate.py` —— `os.statvfs` shim

问题: `os.statvfs` 在 Windows 上**不存在** (`AttributeError`)。CAP-01 容量门禁直接挂。

改造：

```
import shutil  # 新增

try:
    if hasattr(os, "statvfs"):
        stat = os.statvfs(check_path)
        available_bytes = stat.f_bavail * stat.f_frsize
    else:
        available_bytes = shutil.disk_usage(check_path).free
except OSError as exc:
    raise FileSystemError(...)
```

为什么用 `hasattr(os, "statvfs")` 而不是 `sys.platform` 检测：Linux 上即便进了 Docker 容器 `os.statvfs` 也存在；用 `hasattr` 更稳，未来 Mac 也自动走 POSIX 分支。

3.5 `mlpstorage_py/validation_helpers.py` —— 同 3.4

完全同 3.4 的 shim，应用在 `check_disk_space()` 函数上。这是 `mlpstorage` 内部另一处磁盘空间校验点（datagen 前的快速预检）。

4. 环境构建

4.1 装包

```
# 在项目根目录
cd C:\Users\Administrator\Documents\Code\repos\storage

# 1) 锁版本（生成 uv.lock；如果用现成的 lock 文件可跳过）
uv lock

# 2) 装到本地 venv
uv sync
```

预期输出：装 75 个包，**不会出现** `s3dlio` / `s3torchconnector` / `s3torchconnectorclient` 三个名字。

装完验证：

```
& .\venv\Scripts\python.exe -c "import mlpstorage_py; print(mlpstorage_py.__file__)"
& .\venv\Scripts\python.exe -c "import dl原因 benchmark; print(dl原因 benchmark.__file__)"
& .\venv\Scripts\python.exe -c "import s3dlio"
# 最后一条应抛 ModuleNotFoundError（这正是我们想要的）
```

4.2 PowerShell session 环境变量

每个新 PowerShell 窗口都得设一次：

```
# PATH 包含 venv
$env:Path = "$PWD\.venv\Scripts;$env:Path"

# 中文/UTF-8 渲染（否则 dlio_benchmark 输出的希腊字母和 SLO 标记会乱码）
$env:PYTHONIOENCODING = "utf-8"
$env:PYTHONUTF8 = "1"

# MLPerf Storage 强制要求的环境变量（哪怕是 test run 也要有）
$env:MLPERF_SYSTEMNAME = "test_nvme"
```

5. 端到端跑通

5.1 mlpstorage CLI 路径（推荐：先验证“通”，再谈性能）

步骤 1：算数据量

```
mlpstorage closed training unet3d datasize `
  --data-dir D:\mlps_data `
  --results-dir D:\mlps_results
```

步骤 2：生成数据（要 ≥50 GB 可用空间，~10-30 分钟）

```
mlpstorage closed training unet3d datagen `
  --num-processes 4 `
  --data-dir D:\mlps_data `
  --results-dir D:\mlps_results
```

步骤 3：跑训练基准

```
mlpstorage closed training unet3d run `
  --exec-type docker `
  --num-accelerators 2 `
  --accelerator-type h100 `
  --client-host-memory-in-gb 64 `
  --data-dir D:\mlps_data `
  --results-dir D:\mlps_results
```

关于 `--exec-type docker` : Windows 上 `mpirun` + `mpi4py` 链路会触发 `libmpi.dll` 加载的奇怪问题 (PyTorch DataLoader 多进程 fork 时 SemLock 错误) 。用 `docker exec-type` 跳过 MPI 走单进程——这是 Windows 训练路径唯一已验证的稳定跑法。

如果你的 Windows 机器没装 Docker Desktop, 先装; 用 `winget install Docker.DockerDesktop` 或从 docker.com 下。

5.2 dlio_benchmark 直跑路径 (绕过 mlpstorage, 纯存储压测)

如果你只想测 NVMe 的 IO 行为 (不要 MLPerf 包装层) , 直接调 DLIO 引擎:

```
# 一次性生成 50 GB unet3d 数据集
dlio_benchmark workload=unet3d_a100 --workflow.generate_data=True `
  --data-folder=D:\mlps_data\unet3d `
  --output-folder=D:\mlps_results\unet3d_a100

# 跑训练
dlio_benchmark workload=unet3d_a100 `
  --data-folder=D:\mlps_data\unet3d `
  --output-folder=D:\mlps_results\unet3d_a100
```

预期输出 (约 100 秒) :

Metric	Value
AU (Accelerator Utilization)	97.91%
Throughput	10.76 samples/s
IO throughput	1504.5 MiB/s ≈ 1.47 GiB/s
Epochs	5 / 5
Total samples	168

结果位置: `D:\mlps_results\unet3d_a100\*.log`、`*.json` , 其中 `dlio_benchmark.json` 含完整 per-epoch 指标。

6. 关键命令清单（复制粘贴即用）

```
# =====
# 0) 进入项目 & 激活 venv（每开新窗口都跑一次）
# =====
cd C:\Users\Administrator\Documents\Code\repos\storage
$env:Path = "$PWD\.venv\Scripts;$env:Path"
$env:PYTHONIOENCODING = "utf-8"; $env:PYTHONUTF8 = "1"
$env:MLPERF_SYSTEMNAME = "test_nvme"

# =====
# 1) mlpstorage 端到端（推荐先跑这个，验证 shim 没坏）
# =====
mlpstorage closed training unet3d datasize --data-dir D:\mlps_data --results-dir D:\mlps_results
mlpstorage closed training unet3d datagen --num-processes 4 --data-dir D:\mlps_data --results-dir D:\mlps_results
mlpstorage closed training unet3d run --exec-type docker --num-accelerators 2 --accelerator-type docker

# =====
# 2) dlio_benchmark 直跑（纯存储）
# =====
dlio_benchmark workload=unet3d_a100 --data-folder=D:\mlps_data\unet3d --output-folder=D:\mlps_data

# 同样可以跑其他 workload（无需改代码）：
# dlio_benchmark workload=resnet50_a100 ...
# dlio_benchmark workload=cosmoflow_a100 ...

# =====
# 3) 验证 5 个改造点
# =====
git diff --stat
# 应该看到：
#   mlpstorage_py/benchmarks/capacity_gate.py
#   mlpstorage_py/rules/datagen_hierarchy.py
#   mlpstorage_py/submission_checker/tools/code_image.py
#   mlpstorage_py/validation_helpers.py
#   pyproject.toml

# =====
# 4) 一键回滚
# =====
git checkout -- pyproject.toml mlpstorage_py/
```

7. 常见问题 & 排错

Q1. `uv sync` 报 `error: can't find Rust compiler` / `could not find unix in os`

原因：s3dlio 想用 Rust 编译。**解决：**确认 §3.1 的 pyproject 改动已经应用：

```
Select-String pyproject.toml -Pattern "sys_platform"
# 应该看到 sys_platform == 'linux' 和 sys_platform != 'win32'
```

如果还是报，删 lock 重锁：

```
Remove-Item uv.lock
uv lock
uv sync
```

Q2. `mlpstorage training unet3d run` 启动时 `ModuleNotFoundError: s3dlio`

原因：datagen_hierarchy.py 顶层 import 没改。**解决：**确认 §3.2 已经应用，重启 PowerShell 重新激活 venv。

Q3. `mlpstorage` 卡在 "mpirun ... -np 4"，永远不结束

原因：mpi4py 在 Windows + 虚拟环境里 fork DataLoader worker 时 SemLock 错。**解决：**用 `--exec-type docker` 跳过 MPI。

Q4. CAP-01 报 `AttributeError: module 'os' has no attribute 'statvfs'`

原因：§3.4 没改。**解决：**apply 3.4 + 3.5 的 shutil.disk_usage shim。

Q5. `dlio_benchmark` 报 `multiprocessing_context: fork is not supported on Windows`

原因：你在用 `s3dlio_file` / `s3dlio_s3` workload。**解决：**换 `unet3d_a100` / `resnet50_a100` / `cosmoflow_a100` 这些走本地文件系统的 workload。

Q6. `dlio_benchmark` 启动后 5 秒内退出，输出 "Initializing ... Done" 啥都没有

原因：workload 配的 `data_folder` 不存在或为空。**解决：**先跑 `dlio_benchmark ... --workflow.generate_data=True` 生成数据。

Q7. PDF 渲染中文乱码 / pypdf 读出乱码

原因：pypdf 在 CJK PDF 上有 font subsetting bug。**解决：**用 PyMuPDF 验证，或用 Playwright 截图肉眼对比。不要靠 pypdf 抽文字做断言。

Q8. results 目录报 "permission denied" / "文件被占用"

原因：WPS / Excel 还在打开上一个.xlsx。**解决：**

```
Get-Process | Where-Object { $_.Name -match "wps|excel" } | Stop-Process -Force
```

8. 性能基线（本次测试结果）

测试环境： Windows 11 · i7-12700H · 32 GB · 1 TB NVMe 系统盘 (C:) · 288 GB 可用 · DLIO unet3d_a100 · batch_size=7 · epochs=5

Metric	数值	含义
AU (Accelerator Utilization)	97.91 %	训练算子利用率 (> 90% 即"未受存储拖累")
Throughput	10.76 samples/s	端到端训练吞吐
IO throughput	1504.5 MiB/s	实际从盘上读出的带宽 (≈ 1.47 GiB/s)
Total wall clock	~100 s	5 个 epoch 总耗时
Per-epoch samples	168 × 5 = 840	数据集规模
Total bytes read	~25 GB	5 epoch × 168 sample × 140 MB - 缓存命中部分

对比参考（社区里 NVMe 直挂 Linux 的 unet3d_a100 跑分）：

- AU: 94-99%
- IO throughput: 1.2-2.0 GiB/s (依 NVMe 型号)
- → **本机在参考区间内，符合预期**

复现

按 §5.2 直跑路径，结果在 `D:\mlps_results\unet3d_a100\`。看 `dllo_benchmark.json` 里 `metrics.throughput` / `metrics.au` / `metrics.io` 三个字段。

9. 回滚

```
cd C:\Users\Administrator\Documents\Code\repos\storage
git checkout -- pyproject.toml mlpstorage_py/
git status
# 应该回到 5 个文件无 modified 状态
```

lock 文件 (uv.lock)： 你可能也想回滚它 (如果想跑回 Linux 原始 lock)。但要注意，Linux 端如果有别人在用同一个 lock，**回滚 uv.lock 会导致 Linux 端 sync 不到 s3dllo**。建议：**先合**

Linux 端，再在 Windows 端用同一份 lock——或者干脆在 pyproject 用 [tool.uv] environments 列表做 per-platform 区分。

附录 A. 关键文件路径速查

类别	路径
项目根	C:\Users\Administrator\Documents\Code\repos\storage
venv	<项目根>\.venv\
mlpstorage 包	<项目根>\mlpstorage_py\
DLIO workload 模板	<项目根>\configs\dlio\workload*.yaml
DLIO 实际跑的配置	<venv>\Lib\site-packages\dlio_benchmark\configs\workload\unet3d_a100.yaml
测试数据	D:\mlps_data\unet3d\
测试结果	D:\mlps_results\unet3d_a100\
KV cache trace	C:\Users\Administrator\kv_cache_test\trace.csv
测试计划 PDF	<项目根>\docs\AI_SSD_TEST_PLAN.pdf
测试计划 Excel	<项目根>\docs\AI_SSD_TEST_PLAN.xlsx
临时 handoff 文档	%TEMP%\handoff_test_modify.md

附录 B. 用到的环境变量

变量	值	何时设
MLPERF_SYSTEMNAME	test_nvme	mlpstorage 任何子命令都要求
PYTHONIOENCODING	utf-8	中文输出时
PYTHONUTF8	1	Python 3.12 同上
PATH	加 <项目根>\.venv\Scripts	每新 PowerShell 窗口

附录 C. workload 选型速查

想测的	用这个 workload yaml	备注
3D 医学影像, 140MB/sample 随机读	<code>unet3d_a100</code>	本手册主用
通用 CNN, < 1MB/sample 顺序读	<code>resnet50_a100</code>	AU 通常 > 99%
Cosmology 流式读, 混合大小	<code>cosmoflow_a100</code>	适合看读并发
LLM 训练, 巨大 checkpoint	<code>llama_70b.yaml</code>	需 ≥1 TB 空间
LLM 大规模样本读	<code>dlrm.yaml</code>	适合看元数据 IO
不要在 Windows 上跑	<code>*_s3.yaml</code> / <code>*_s3dlio_*</code>	强制要 s3dlio, 编译不过

最后更新 2026-08-02 · 适用 mlpstorage v3.0.46

下一步 跑通后建议把这份手册和 `AI_SSD_TEST_PLAN.pdf` 一起发给团队评审, 再决定是否要 cherry-pick 这 5 个改动到主分支。